TECNOLOGÍA EN DESARROLLO DE SOFTWARE

PROGRAMACIÓN AVANZADA Y APLICADA

PROYECTO ASISTENTEIA

ARQUITECTURA

Estilo arquitectónico, componentes y flujos

Elaborado por	Juan Esteban Olaya Muñoz
Docente	Mg. Sandra Esperanza Sánchez Barragán
Programa	Tecnología en Desarrollo de Software
Materia	Programación Avanzada y Aplicada
Modalidad	Individual
Producto	Estilo arquitectónico, componentes y flujos
Fecha	23/10/2026

1. Referencia conceptual

Se usó como referencia el material de la actividad “Tipos de Arquitectura de Software” (monolítica, en capas, cliente-servidor, MVC, microservicios, orientada a eventos y serverless), que compara cómo se organiza cada una, sus ventajas, limitaciones y casos de uso, y señala que una solución puede combinar estilos. El ejemplo de comercio electrónico del material no se traslada a AsistenteIA.

2. Estilos evaluados

Estilo	Cómo se organiza	Ventaja	Limitación	Decisión
En capas / MVVM	Presentación (Views), lógica/servicios (ViewModels + Core) y acceso a datos (repositorio, almacenes).	Separa responsabilidades y facilita pruebas; sencillo para una app móvil.	Demasiadas capas añaden rigidez; aquí se mantienen pocas.	Adoptado para la organización interna.
Cliente-servidor	La app es cliente; el proveedor de IA es el servidor mediante HTTPS/REST.	Delegar el procesamiento de IA al proveedor; contrato JSON estándar.	Dependencia de red, límites y costos del proveedor.	Adoptado para la comunicación con la IA.
Microservicios	Servicios independientes por capacidad de negocio con despliegue propio.	Escalado y despliegue independientes en sistemas grandes.	Complejidad operativa, red y observabilidad; desproporcionado aquí.	Descartado: no existe backend propio, la app solo consume una API externa.

3. Estilo adoptado y justificación

AsistenteIA combina cliente-servidor (app MAUI ↔ API REST externa) con una organización interna por capas/MVVM. No existe backend propio ni microservicios. La elección responde al alcance real (una app móvil con un solo flujo de IA), la simplicidad, la facilidad de pruebas (la lógica vive en AsistenteIA.Core, probada sin UI), la seguridad (la clave nunca sale del dispositivo salvo hacia el proveedor elegido) y el bajo costo de mantenimiento.

4. Diagrama de arquitectura

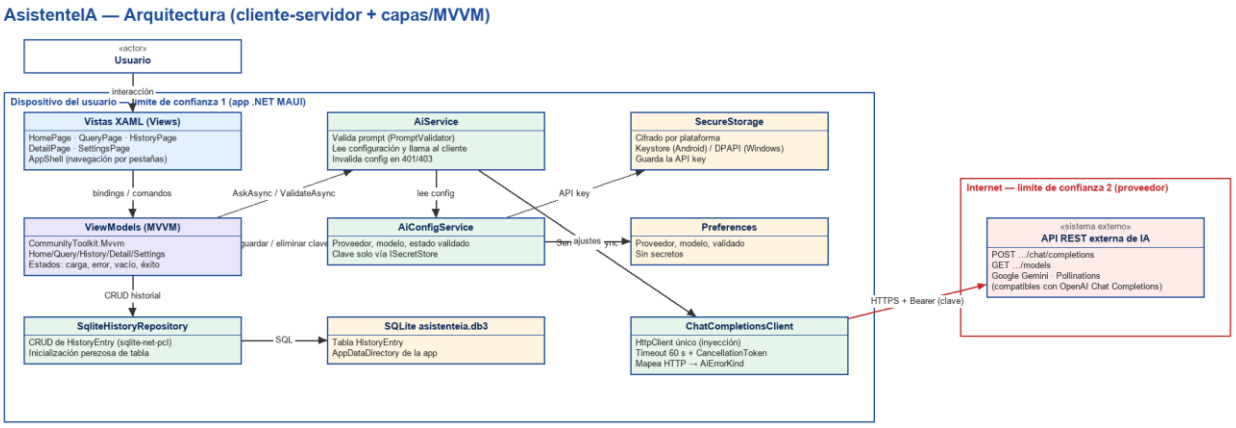


Figura 1. Componentes, límites de confianza y flujo de datos (editable: DIAGRAMA_ARQUITECTURA.drawio).

5. Componentes y responsabilidades

Componente	Responsabilidad
Views (XAML)	Presentación y bindings; sin lógica de negocio. Cinco páginas + MessageBanner + AppShell.
ViewModels	Estado de pantalla y comandos (CommunityToolkit.Mvvm). Orquestan servicios.
AiService	Valida el prompt, obtiene configuración y llama al cliente REST.
AiConfigService	Persiste proveedor/modelo/validación y la clave mediante ISecretStore.
ChatCompletionsClient	Solicitudes HTTP al proveedor, serialización JSON tipada, timeout y mapeo de errores.
SqliteHistoryRepository	CRUD del historial en SQLite con inicialización segura.
SecureStorage / Preferences	Almacén cifrado de la clave / ajustes sin secretos.

6. Flujos principales

6.1 Configurar y validar la clave

1. El usuario pega la clave (campo oculto) y pulsa Detectar proveedor (detección local por prefijo) o elige proveedor.
2. Pulsa Probar conexión/Guardar: AiService envía una solicitud de 1 token solo al proveedor elegido.
3. Si hay éxito se guarda la clave en SecureStorage y el estado “validado” en Preferences.

6.2 Consultar y guardar

4. El usuario escribe el prompt; PromptValidator lo valida.
5. AiService → ChatCompletionsClient → POST chat/completions.
6. La respuesta se muestra; si el usuario guarda, se inserta un HistoryEntry en SQLite.

7. Límites de confianza

- Límite 1 (dispositivo): SecureStorage protege la clave en reposo; un dispositivo comprometido o con root puede exponerla.
- Límite 2 (internet): la clave viaja solo por HTTPS al proveedor elegido; el contenido del prompt es visible para el proveedor.

8. Referencias consultadas

- Material de la actividad «Tipos de Arquitectura de Software» (Tecnología en Desarrollo de Software): comparación de arquitecturas monolítica, en capas, cliente-servidor, MVC, microservicios, orientada a eventos y serverless.
- Pollinations. Documentación de la API: https://gen.pollinations.ai/
- Google AI for Developers. Gemini API: https://ai.google.dev/gemini-api/docs